# 10 Clojure: A different view of programming

This chapter covers

- Clojure's concept of identity and state
- The Clojure REPL
- Clojure syntax, data structures, and sequences
- Clojure interoperability with Java
- Clojure macros

Clojure is a very different style of language from Java and the other languages we've studied so far. Clojure is a JVM reboot of one of the oldest programming languages—Lisp. If you're not familiar with Lisp, don't worry. We'll teach you everything you need to know about the Lisp family of languages to get you started with Clojure.

**NOTE** Because Clojure is such a different language, it might help to have an additional, Clojure-specific resource to consult while reading this chapter. A couple of excellent books are *Clojure in Action* (Manning, 2011; **https://livebook.manning.com/book/clojure-in-action**) and *The Joy of Clojure* (Manning, 2014; **https://livebook.manning.com/book/the-joy-of-clojure-second-edition**).

In addition to its heritage of powerful programming techniques from classic Lisp, Clojure adds amazing cutting-edge technology that's very relevant to the modern Java developer. This combination makes Clojure a standout language on the JVM and an attractive choice for application development. Particular examples of Clojure's new tech are its concurrency toolkits (which we will meet in chapter 16) and data structures (which we will introduce here and expand on in chapter 15).

For the avid reader who can't wait until later, let us just say this: the concurrency abstractions enable programmers to write much safer multithreaded code than when working in Java. These abstractions can be combined with Clojure's seq concept (a different take on collections and data structures) to provide a powerful developer toolbox.

To access all of this power, some important language concepts are approached in a fundamentally different way from Java. This difference in approach makes Clojure interesting to learn, and it will probably also change the way you think about programming.

**NOTE** Learning Clojure will help make you a better programmer in any language. Functional programming matters.

We'll kick off with a discussion of Clojure's approach to state and variables. After some simple examples, we'll introduce the basic vocabulary of the language—the *special forms* that are equivalent to keywords in languages like Java. A small number of these are used to build up the rest of the language.

We'll also delve into Clojure's syntax for data structures, loops, and functions. This will allow us to introduce sequences, which are one of Clojure's most powerful abstractions.

We'll conclude the chapter by looking at two very compelling features: tight Java integration and Clojure's amazing macro support (which is the key to Lisp's very flexible syntax). Later in the book, we'll meet more Clojure goodness (as well as Kotlin and Java examples) when we talk about advanced functional programming (chapter 15) and advanced concurrency (chapter 16).

## 10.1 Introducing Clojure

The basic unit of Lisp syntax consists of an expression to be evaluated. These expressions are typically represented as zero or more symbols surrounded by brackets. If the evaluation succeeds without errors, the expression is called a *form*.

**NOTE** Clojure is compiled, not interpreted, but the compiler is very simple. Also remember that Clojure is dynamically typed, so there won't be many type-checking errors to help you—they will show up as runtime exceptions instead.

Simple examples of forms include:

```
0
(+ 3 4)
(list 42)
(quote (a b c))
```

The true core of the language has very few built-in forms (the special forms). They are the Clojure equivalent of Java keywords, but be aware of the following:

1. Clojure has a different meaning for the term *keyword*, which we'll encounter later.
2. Clojure (like all Lisps) allows the creation of constructs that are indistinguishable from built-in syntax.

When working with Clojure code, it almost never matters whether the forms you're using are special forms or library functions that are built up from them.

Let's get started with forms by looking at one of Clojure's most important conceptual differences from Java. This is the treatment of state, variables, and storage. As you can see in figure 10.1, Java (like Kotlin) has a model of memory and state that involves a variable being a "box" (really, a memory location) with contents that can change over time.

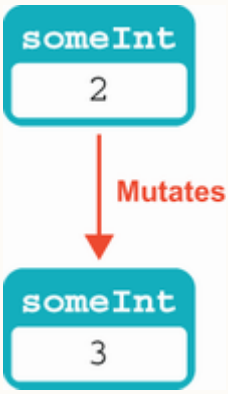



Figure 10.1 Imperative language memory use

Programming languages like Java are *mutable by default*, because we are trying to alter the program state, which in Java is made up of objects. Languages that follow this model are often called imperative languages, as we discussed in chapter 8.

Clojure is a little bit different. The important concept is that of a *value*. Values can be numbers, strings, vectors, maps, sets, or a number of other things. Once created, values never alter. This is really important, so we'll say it again: *once created, Clojure values can't be altered*—they're *immutable*.

**NOTE** Immutability is a common property of languages that are used for functional programming, because it allows mathematical reasoning techniques about the properties of functions (such as the same input always giving the same output) to be used.

The imperative language model of a box that has contents that change isn't the way Clojure works. Figure 10.2 shows how Clojure deals with state and memory. It creates an association between a name and a value.

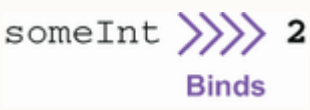



Figure 10.2 Clojure memory use

This is called *binding*, and it's done using the `def` special form. Let's meet the syntax for `(def)` here:

```
(def <name> <value>)
```

Don't worry that the syntax looks a little weird—this is entirely normal for Lisp syntax, and you'll get used to it really quickly. For now you can pretend that the brackets are arranged slightly differently and that you're calling a method like this:

```
def(<name>, <value>)
```

Let's demonstrate `(def)` with a time-honored example that uses the Clojure interactive environment.

### 10.1.1 Hello World in Clojure

If you haven't already installed Clojure, you can do so on a Mac by running this command:

```
brew install clojure/tools/clojure
```

This will install the command-line tools with brew from the `clojure/tools` tap. For other operating systems, instructions can be found on the **clojure.org** website.

**NOTE** Windows support isn't so great for Clojure. For example, `clj` is still in an alpha state. Follow the instructions on the website carefully.

Once installed, you can use the `clj` command to start the Clojure interactive session. Or, if you built Clojure from source, change into the directory where you installed Clojure and run this command:

```
java -cp clojure.jar clojure.main
```

Either way, this brings up the user prompt for the Clojure read-evaluate-print loop (REPL). This is the interactive session, which is where you'll typically spend quite a lot of time when developing Clojure code. It looks like this:

```
$ clj
Clojure 1.10.1
user=>
```

The `user=>` part is the Clojure prompt for the session, which can be thought of as a bit like an advanced debugger or a command line. To exit the session (which will cause all the accumulated state in the session to be lost), use the traditional Unix sequence `Ctrl-D`. Let's write a "Hello World" program in Clojure:

```
user=> (def hello (fn [] "Hello world"))
#'user/hello

user=> (hello)
"Hello world"
user=>
```

In this code, you start off by binding the *identifier* `hello` to a value. `(def)` always binds identifiers (which Clojure calls *symbols*) to *values*. Behind the scenes, it will also create an object, called a *var*, that represents the binding (and the name of the symbol), as shown next:

```
(def hello (fn [] "Hello world"))
 --- ----- --------------------
  |   |           |
  |   |         value
  | symbol
  |
special form
```

What is the value you're binding `hello` to? It's the value

```
(fn [] "Hello world")
```

This is a function, which is a genuine value (so, therefore, immutable) in Clojure. It's a function that takes no arguments and returns the string "Hello world". The empty argu-

ment list is represented by the `[]`.

**NOTE** In Clojure (but not in other Lisps), square brackets indicate a linear data structure called a vector—in this case, a vector of function arguments.

After binding it, you execute it via `(hello)`. This causes the Clojure runtime to print the results of evaluating the function, which is "Hello world".

Remember that the round brackets mean "function evaluation" in Lisps, so the example basically consists of the following:

- Create a function, and bind it to the symbol `hello`.
- Call the function bound to the symbol `hello`.

At this point, you should enter the Hello World example (if you haven't already) and see that it behaves as described. Once you've done that, we can explore a little further.

### 0.1.2 Getting started with the REPL

The REPL allows you to enter Clojure code and execute Clojure functions. It's an interactive environment, and the results of earlier evaluations are still around. This enables a type of programming called *exploratory programming*, which basically means that you can experiment with code. In many cases the right thing to do is to play around in the REPL, building up larger and larger functions once the building blocks are correct.

**NOTE** Subdivision is a key technique in functional programming—breaking down a problem into smaller parts until it becomes either soluble or amenable to a reusable pattern (which may already be in the standard library).

Let's look at a bit more Clojure syntax. One of the first things to point out is that the binding of a symbol to a value can be changed by another call to `(def)`, so let's see that in action in the REPL. We'll actually use a slight variant of `(def)` called `(defn)`, as follows:

```
user=> (hello)
"Hello world"

user=> (defn hello [] "Goodnight Moon")
#'user/hello
```

```
user=> (hello)
"Goodnight Moon"
```

Notice that the original binding for hello is still in play until you change it—this is a key feature of the REPL. There is still state, in terms of which symbols are bound to which values, and that state persists between lines the user enters.

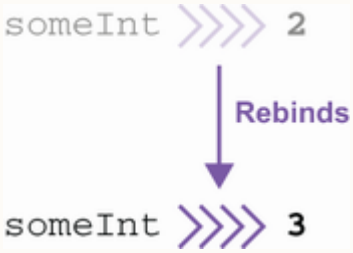



Figure 10.3 Clojure bindings changing over time

The ability to change which value a symbol is bound to is Clojure's alternative to mutating state. Rather than allowing the contents of a storage location (or "memory box") to change over time, Clojure allows a symbol to be bound to different immutable values at different points in time. Another way of saying this is that the var can point to different values during the lifetime of a program. An example can be seen in figure 10.3.

**NOTE** This distinction between mutable state and different bindings at different times is subtle, but it's an important concept to grasp. Remember, *mutable state* means the contents of the box change, whereas *rebinding* means pointing at different boxes at different points in time.

This is in some ways similar to the Java concept of `final` references. In Java, if we say `final int`, the contents of the storage location cannot change. As `int`s are stored as bit patterns, which means that the value of the `int` cannot change.

However, if we say `final AtomicInteger`, the contents of the storage location once again cannot change. This case is different, though, because a variable containing an atomic integer actually holds an object reference. The atomic integer object stored in the heap can change the value it stores (whereas an `Integer` cannot), and this is true whether or not the reference to the object is final.

We've also slipped in another Clojure concept in the last code snippet—the `(defn)` "define function" *macro*. Macros are one of the key concepts of Lisp-like languages. The central idea is that there should be as little distinction between built-in constructs and ordinary code as possible.

**NOTE** Macros allow you to create forms that behave like built-in syntax. The creation of macros is an advanced topic, but mastering their creation will allow you to produce incredibly powerful tools.

The true language primitives of the system (the special forms) can be used to build up the core of the language in such a way that you don't really notice the difference between the two.

**NOTE** The `(defn)` macro is an example of this. It's just a slightly easier way to bind a function value to a symbol (and create a suitable var, of course). It's not a special form but instead is a macro built up from the special forms `(def)` and `(fn)`.

We will introduce macros properly at the end of this chapter.

### 0.1.3 Making a mistake

What happens if you make a mistake? Suppose you're trying to declare a function but accidentally just `def` a value instead, like this:

```
user=> (def hello "Goodnight Moon")
#'user/hello

user=> (hello)
Execution error (ClassCastException) at user/eval137 (REPL:1).
class java.lang.String cannot be cast to class clojure.lang.IFn
(java.lang.String is in module java.base of loader 'bootstrap';
clojure.lang.IFn is in unnamed module of loader 'app')
```

There's a couple of things to notice here. First is that the error is a runtime exception. This means that the form `(hello)` compiled fine; it just failed at runtime. In terms of the equivalent code in Java, it looks a bit like this (we've simplified things somewhat to make it easier to understand for folks who are new to Clojure or language implementation):

```
// (def hello "Goodnight Moon")
var helloSym = Symbol.of("user", "hello");
var hello = Var.of(helloSym, "Goodnight Moon");

// Or just
// var hello = Var.of(Symbol.of("user", "hello"), "Goodnight Moon");

// #'user/hello
```

```
// (hello)
hello.invoke();

// ClassCastException
```

where `Symbol` and `Var` are classes in the package `clojure.lang` that provides the core of the Clojure runtime. They look similar to these basic implementations, which we have simplified here:

```
public class Symbol {
   private final String ns;
   private final String name;

   private Symbol(String ns, String name) {
      this.ns = ns;
      this.name = name;
   }
   // toString() etc
}

public class Var implements IFn {
   private volatile Object root;

   public final Symbol sym;
   public final Namespace ns;

   private Var(Symbol sym, Namespace ns, Object root) {
      this.sym = sym;
      this.ns = ns;
      this.root = root;
   }

   public static Var of(Symbol sym, Object root){
      return new Var(sym, Namespace.of(sym), root);
   }

   static public class Unbound implements IFn {
      final public Var v;
      public Unbound(Var v){
         this.v = v;
      }

      @Override
```

```
        public String toString(){
            return "Unbound: " + v;
        }
    }

    public synchronized void bindRoot(Object root) {
        this.root = root;
    }

    public synchronized void unBindRoot(Object root) {
        this.root = new Unbound(this);
    }

    @Override
    public Object invoke() {
        return ((IFn)root).invoke();
    }

    @Override
    public Object invoke(Object o1) {
        return ((IFn)root).invoke(o1);
    }

    @Override
    public Object invoke(Object o1, Object o2) {
        return ((IFn)root).invoke(o1, o2);
    }

    @Override
    public Object invoke(Object o1, Object o2, Object o3) {
        return ((IFn)root).invoke(o1, o2, o3);
    }
    // ...
}
```

The all-important interface `IFn` looks a bit like this:

```
public interface IFn {
    default Object invoke() {
        return throwArity();
    }
    default Object invoke(Object o1) {
        return throwArity();
    }
    default Object invoke(Object o1, Object o2) {
```

```
        return throwArity();
    }
    default Object invoke(Object o1, Object o2, Object o3) {
        return throwArity();
    }

    // ... many others including eventually a variadic form

    default Object throwArity(){
        throw new IllegalArgumentException("Wrong number of args passed: "
                + toString());
    }
}
```

`IFn` is the key to how Clojure forms work—the first element in a form is taken to be a function, or the name of a function, to be invoked. The remaining elements are the arguments to the function, and the `invoke()` method with the appropriate number of arguments (arity) is called.

If a Clojure var is not bound to a value that implements `IFn`, a `ClassCastException` is thrown at runtime. If the value is an `IFn` but the form tries to invoke it with the wrong number of arguments, an `IllegalArgumentException` is thrown (it's actually a subtype called an `ArityException`).

**NOTE** Remember that Clojure is dynamically typed, as you can see in several places, for example, all the arguments and return types of the methods in `IFn` are `Object` and `IFn` is *not* a Java-style `@FunctionalInterface` but instead has multiple methods defined on it to handle many different arities.

This peek under the hood should help clarify both a little of Clojure's syntax and how it all fits together. However, we still have some broken code to fix—but fortunately it's not too hard!

All that's happened is that you've got your `hello` identifier bound to something that isn't a function so it can't be called. In the REPL, you can fix this by simply rebinding it like so:

```
user=> (defn hello [] (println "Dydh da an Nor")) ; "Hello World" in Cornish
#'user/hello

user=> (hello)
```

```
Dydh da an Nor
nil
```

As you might guess from the preceding snippet, the semicolon ( `;` ) character means that everything to the end of the line is a comment, and `(println)` is the function that prints a string. Notice that `(println)`, like all functions, returns a value, which is echoed back to the REPL at the end of the function's execution.

Clojure does not have statements like Java, only expressions, so all functions must return a value. If there is no value to return, then `nil` is used, which is basically the Clojure equivalent of Java's `null`. Functions that would be `void` in Java will return `nil` in Clojure.

### 0.1.4 Learning to love the brackets

The culture of programmers has always had a large element of whimsy and humor. One of the oldest jokes is that Lisp is an acronym for *Lots of Irritating Silly Parentheses* (instead of the more prosaic truth—that it's an abbreviation for *list processing*). This rather self-deprecating joke is popular with some Lisp coders, partly because it points out the unfortunate truth that Lisp syntax has a reputation for being difficult to learn.

In reality, this hurdle is rather exaggerated. Lisp syntax is different from what most programmers are used to, but it isn't the obstacle that it's sometimes presented as. In addition, Clojure has several innovations that reduce the barrier to entry even further.

Let's take another look at the Hello World example. To call the function that returns the value "Hello World", we wrote this:

```
(hello)
```

If we want functions with arguments, rather than having expressions such as `myFunction(someObj)`, in Clojure we write `(myFunction someObj)`. This syntax is called *Polish notation*, because it was developed by Polish mathematicians in the early 20th century (it is also called *prefix notation*).

If you've studied compiler theory, you might wonder whether there's a connection here to concepts like the abstract syntax tree (AST). The short answer is yes, there is. A Clojure (or other Lisp) program that is written in Polish notation (usually called an s-*ex*-

*pression* by Lisp programmers) can be shown to be a very simple and direct representation of the AST of that program.

**NOTE** This relates back, once again, to the simple nature of the Clojure compiler. Compilation of Lisp code is a very cheap operation, because the structure is so close to the AST.

You can think of a Lisp program as being written in terms of its AST directly. There's no real distinction between a data structure representing a Lisp program and the code, so code and data are very interchangeable. This is the reason for the slightly strange notation: it's used by Lisp-like languages to blur the distinction between built-in primitives and user and library code. This power is so great that it far outweighs the slight oddity of the syntax to the eyes of a newly arrived Java programmer. Let's dive into some more of the syntax and start using Clojure to build real programs.

## 10.2 Looking for Clojure: Syntax and semantics

In the previous section, you met the `(def)` and `(fn)` special forms (we also met `(defn)`, but it's a macro, not a special form). You need to know a small number of other special forms immediately to provide a basic vocabulary for the language. In addition, Clojure offers a large number of useful forms and macros, of which a greater awareness will develop with practice.

Clojure is blessed with multiple useful functions for doing a wide range of conceivable tasks. Don't be daunted by this— embrace it. Be happy that for many practical programming tasks you may face in Clojure, somebody else has already done the heavy lifting for you.

In this section, we'll cover the basic working set of special forms, then progress to Clojure's native data types (the equivalent of Java's collections). After that, we'll progress to a natural style for writing Clojure—one in which functions rather than variables have center stage. The object-oriented nature of the JVM will still be present beneath the surface, but Clojure's emphasis on functions has a power that is not as obviously present in purely OO languages and which goes far beyond the basics of `map()`, `filter()`, and `reduce()`.

### 10.2.1 Special forms bootcamp

Table 10.1 covers the definitions of some of Clojure's most commonly used special forms. To get best use of the table, skim through it now and refer back to it as necessary when you reach some of the examples in sections 10.3 onward. The table uses the traditional

regular expression syntax notation where `?` represents a single optional value and `*` represents zero or more values.

This isn't an exhaustive list of special forms, and a high percentage of them have multiple ways of being used. Table 10.1 is a starter collection of basic use cases and not anything comprehensive.

Table 10.1 Some of Clojure's basic special forms

| Special form | Meaning |
| --- | --- |
| `(def <symbol> <value?>)` | Binds a symbol to a value (if provided); creates a var corresponding to the symbol if necessary |
| `(fn <name>? [<arg>*] <expr>*)` | Returns a function value that takes the specified args and applies them to the exprs; often combined with `(def)` into forms like `(defn)` |
| `(if <test> <then> <else>?)` | If test evaluates to logical-true, evaluate and yield then; otherwise, evaluate and yield else, if present |
| `(do <expr>*)` | Evaluates the exprs in left-to-right order and yields the value of the last |
| `(let [<binding>*] <expr>*)` | Aliases values to a local name and implicitly defines a scope; makes the alias available inside all exprs within the scope of let |
| `(quote <form>)` | Returns form as is without evaluating anything; takes a single form and ignores all other arguments |
| `(var <symbol>)` | Returns the var corresponding to symbol (returns a Clojure JVM object, not a value) |

A couple of points deserve further explanation, because the structure of Clojure code can seem very different to Java code at first glance. First, the `(do)` form is one of the simplest ways to construct what would be a block of statements in Java.

Second, we need to dig a bit deeper into the distinction between a var, a value, and the symbol that a value is (temporarily) bound to. This simple code creates a Clojure var called `hi`. This is a JVM object (an instance of the type `clojure.lang.Var`) that lives in the heap—as all objects do—and binds it to a `java.lang.String` object containing "hello":

```
user=> (def hi "Hello")
#'user/hi
```

The var has a *symbol* `hi`, and it also has a *namespace* `user` that Clojure uses to organize programs—a bit like a Java package. If we use the symbol unadorned in the REPL, it evaluates to the value it is currently bound to, as shown here:

```
user=> hi
"Hello"
```

In the `(def)` form, we bind a new symbol to a value, so in this code

```
user=> (def bye hi)
#'user/bye
```

the symbol `bye` is bound to the *value* currently bound to `hi`, as shown next:

```
user=> bye
"Hello"
```

Effectively, in this simple form, `hi` is evaluated and the symbol is replaced with the value that results.

However, Clojure offers us more possibilities than just this. For example, the value that a symbol is bound to is just any JVM value. So, we can bind a symbol to the var we have created because the var is itself a JVM object. This is achieved using the `(var)` special form as shown here:

```
user=> (def bye (var hi))
#'user/bye
```

```
user=> bye
#'user/hi
```

This effectively uses the fact that Java/JVM objects are always handled by reference, as we can see in figure 10.4.

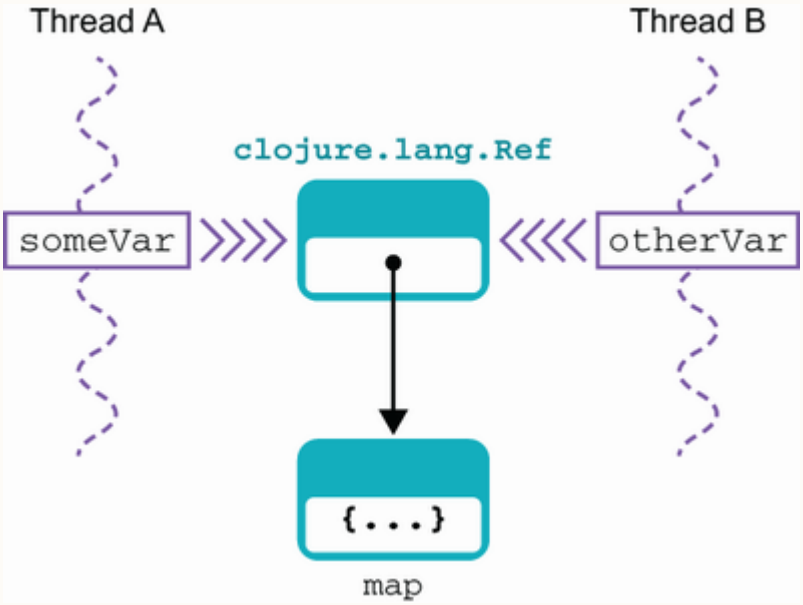



Figure 10.4 Clojure var acting by reference

To get back the value contained in a var, we can use the `(deref)` form (short for "dereference"), like this:

```
user=> (deref bye)
"Hello"
```

There is also a `(ref)` form that is used for safe concurrent programming in Clojure—we will meet it in chapter 16.

From this distinction between a var and the value it is currently bound to, the `(quote)` form should be easier to understand. Instead of evaluating the form it is passed, it simply returns a form comprising the unevaluated symbols.

Now that you have an appreciation of the syntax for some basic special forms, let's turn to Clojure's data structures and start to see how the forms can operate on data.

### 0.2.2 Lists, vectors, maps, and sets

Clojure has several native data structures. The most familiar is the *list*, which in Clojure is a singly linked list.

**NOTE** In some respects, a Clojure list is similar to a `LinkedList` in Java, except that `LinkedList` is a doubly-linked list where each element has a reference to both the next element and the previous one.

Lists are typically surrounded with parentheses, which seemingly presents a slight syntactic hurdle because round brackets are also used for general forms. In particular, parentheses are used for evaluation of function calls. This leads to the following common beginner's syntax error:

```
user=> (1 2 3)
Execution error (ClassCastException) at user/eval1 (REPL:1).
class java.lang.Long cannot be cast to class clojure.lang.IFn
(java.lang.Long is in module java.base of loader 'bootstrap';
clojure.lang.IFn is in unnamed module of loader 'app')
```

The problem here is that, because Clojure is very flexible about its values, it's expecting a function value (or a symbol that resolves to one) as the first argument, so it can call that function and pass 2 and 3 as arguments; 1 isn't a value that is a function, so Clojure can't evaluate this form. We say that this *s* -expression is invalid, and recall that only valid *s* -expressions are Clojure forms.

The solution is to use the `(quote)` form that we met in the previous section. This has a handy short form, which is `'`. This gives us these two equivalent ways of writing this list, which consists of the immutable list of three elements that are the numbers 1, 2 and 3, as follows:

```
user=> '(1 2 3)
(1 2 3)

user=> (quote (1 2 3))
(1 2 3)
```

Note that `(quote)` handles its arguments in a special way. In particular, there is no attempt made to evaluate the argument, so there's no error arising from a lack of a function value in the first slot.

Clojure has vectors, which are like arrays (in fact, it's not too far from the truth to think of lists as being basically like Java's `LinkedList` and vectors as like `ArrayList`). They have a convenient literal form that makes use of square brackets, so all of the following are equivalent:

```
user=> (vector 1 2 3)
[1 2 3]

user=> (vec '(1 2 3))
[1 2 3]

user=> [1 2 3]
[1 2 3]
```

We've already met vectors. When we declared the Hello World function and others, we used a vector to indicate the parameters that the declared function takes. Note that the form `(vec)` accepts a list and creates a vector from it, whereas `(vector)` is a form that accepts multiple individual symbols and returns a vector of them.

The function `(nth)` for collections takes two parameters: a collection and an index. It can be thought of as similar to the `get()` method from Java's `List` interface. It can be used on vectors and lists, and also on Java collections and even strings, which are treated as collections of characters. Here's an example:

```
user=> (nth '(1 2 3) 1)
2
```

Clojure also supports maps (which you can think of as being very similar to Java's `HashMap`—and they do in fact implement the `Map` interface) with this simple literal syntax:

```
{key1 value1 key2 value2}
```

To get a value back out of a map, the syntax, shown next, is very simple:

```
user=> (def foo {"aaa" "111" "bbb" "2222"})
#'user/foo

user=> foo
{"aaa" "111", "bbb" "2222"}

user=> (foo "aaa")          ❶
"111"
```

❶ This syntax is equivalent to the use of a get() method in Java.

In addition to the `Map` interface, Clojure maps also implement the `IFn` interface, which is why they can be used in a form like `(foo "aaa")` without a runtime exception.

One very useful stylistic point is the use of keys that have a colon in front of them. Clojure refers to these as *keywords*.

**NOTE** The Clojure usage of "keyword" is, of course, very different from the meaning of that term in other languages (including Java) where the term means the parts of the language grammar that are reserved and not able to be used as identifiers.

Here are some useful points about keywords and maps to keep in mind:

- A keyword in Clojure is a function that takes one argument, which must be a map.
- Calling a keyword function on a map returns the value that corresponds to the keyword function in the map.
- When using keywords, there's a useful symmetry in the syntax, as `(my-map :key)` and `(:key my-map)` are both legal.
- As a value, a keyword returns itself.
- Keywords don't need to be declared or `def`'d before use.
- Remember that Clojure functions are values and, therefore, are eligible to be used as keys in maps.
- Commas can be used (but aren't necessary) to separate key-value pairs, because Clojure considers them whitespace.
- Symbols other than keywords can be used as keys in Clojure maps, but the keyword syntax is extremely useful and is worth emphasizing as a style in your own code.

Let's see some of these points in action here:

```
user=> (def martijn {:name "Martijn Verburg", :city "London",
:area "Finsbury Park"})
#'user/martijn

user=> (:name martijn)    ❶
"Martijn Verburg"

user=> (martijn :area)    ❷
"Finsbury Park"

user=> :area              ❸
:area
```

```
user=> :foo          ❸
:foo
```

❶ Calls the keyword function on the map

❷ Looks up the value associated to the keyword in the map

❸ Shows that when evaluated as a value, a keyword returns itself

In addition to map literals, Clojure also has a `(map)` function. But don't be caught out. Unlike `(list)`, the `(map)` function doesn't produce a map. Instead, `(map)` applies a supplied function to each element in a collection in turn and builds a new collection (actually a Clojure sequence, which you'll meet in detail in section 10.4) from the new values returned. This is, of course, the Clojure equivalent to the `map()` method that you have already met from Java's Streams API, shown here:

```
user=> (def ben {:name "Ben Evans", :city "Barcelona", :area
"El Born"})
#'user/ben

user=> (def authors [ben martijn])      ❶
#'user/authors

user=> (defn get-name [y] (:name y))
#'user/get-name

user=> (map get-name authors)           ❷
("Ben Evans" "Martijn Verburg")

user=> (map (fn [y] (:name y)) authors)  ❸
("Ben Evans" "Martijn Verburg")
```

❶ Creates a vector of maps of author data

❷ Maps the get-name function over the data

❸ Alternates form using an inline function literal

There are additional forms of `(map)` that are able to handle multiple collections at once, but the form that takes a single collection as input is the most common.

Clojure also supports sets, which are very similar to Java's `HashSet`. They have a short form for data structure literals that do not support repeated keys (unlike `HashSet`), shown here:

```
user=> #{"a" "b" "c"}
#{"a" "b" "c"}

user=> #{"a" "b" "a"}
Syntax error reading source at (REPL:15:15).
Duplicate key: a
```

These data structures provide the fundamentals for building up Clojure programs.

One thing that may surprise the Java native is the lack of any immediate mention of objects as first-class citizens. This isn't to say that Clojure isn't object-oriented, but it doesn't see OO in quite the same way as Java. Java chooses to see the world in terms of statically typed bundles of data and code in explicit class definitions of user-defined data types. Clojure emphasizes the functions and forms instead, although these are implemented as objects on the JVM behind the scenes.

This philosophical distinction between Clojure and Java manifests itself in how code is written in the two languages, and to fully understand the Clojure viewpoint, it's necessary to write programs in Clojure and understand some of the advantages that deemphasizing Java's OO constructs brings.

### 0.2.3 Arithmetic, equality, and other operations

Clojure has no operators in the sense that you might expect them in Java. So, how would you, for example, add two numbers? In Java it's easy:

```
3 + 4
```

But Clojure has no operators. We'll have to use a function instead, as follows:

```
(add 3 4)  ❶
```

❶ This code won't work as it stands, unless we supply an add function.

That's all well and good, but we can do better. Because there aren't any operators in Clojure, we don't need to reserve any of the keyboard's characters to represent them. That means our function names can be more outlandish than in Java, so we can write this:

```
(+ 3 4)  ❶
```

❶ This is literally Polish notation, as discussed earlier.

Clojure's functions are in many cases *variadic* (they take a variable number of inputs), so you can, for example, write this:

```
(+ 1 2 3)
```

This will give the value 6.

For the equality forms (the equivalent of `equals()` and `==` in Java), the situation is a little more complex. Clojure has two main forms that relate to equality: `(=)` and `(identical?)`. Note that these are both examples of how the lack of operators in Clojure means that more characters can be used in function names. Also, `(=)` is a single equals sign, because there's not the same notion of assignment as in Java-like languages.

This bit of REPL code sets up a list, `list-int`, and a vector, `vect-int`, and applies equality logic to them like so:

```
user=> (def list-int '(1 2 3 4))
#'user/list-int

user=> (def vect-int (vec list-int))
#'user/vect-int

user=> (= vect-int list-int)
true

user=> (identical? vect-int list-int)
false
```

The key point is that the `(=)` form on collections checks to see whether the collections comprise the same objects in the same order (which is true for `list-int` and `vect-int`),

whereas `(identical?)` checks to see whether they're really the same object.

You might also notice that our symbol names don't use camel case. This is usual for Clojure. Symbols are usually all in lowercase, with hyphens between words (sometimes called *kebab case*).

#### True and false in Clojure

Clojure provides two values for logical false: `false` and `nil`. Anything else is logical true (including the literal `true`). This parallels the situation in many dynamic languages (e.g., JavaScript), but it's a bit strange for Java programmers encountering it for the first time.

With basic data structures and operators under our belts, let's put together some of the special forms and functions we've seen and write slightly longer example Clojure functions.

### 10.2.4 Working with functions in Clojure

In this section, we'll start dealing with some of the meat of Clojure programming. We'll start writing functions to act on data and bring Clojure's focus on functions to the fore. Next up are Clojure's looping constructs, then reader macros and dispatch forms. We'll round out the section by discussing Clojure's approach to functional programming and its take on closures.

The best way to start doing all of this is by example, so let's get going with a few simple examples and build up toward some of the powerful functional programming techniques that Clojure provides.

#### Some simple Clojure functions

The next listing defines three functions, two of which are very simple functions of one argument; the third is a little more complex.

**Listing 10.1 Defining simple Clojure functions**

```
(defn const-fun1 [y] 1)

(defn ident-fun [y] y)

(defn list-maker-fun [x f]      ❶
  (map (fn [z] (let [w z]       ❷
```

```
    (list w (f w))          ❸
  )) x))
```

❶ The list maker takes two arguments, the second of which is a function.

❷ An inline, anonymous function

❸ Makes a list of two elements: the value and the result of applying f to the value

In this listing, `(const-fun1)` takes in a value and returns 1, and `(ident-fun)` takes in a value and returns the very same value. Mathematicians would call these a *constant function* and the *identity function*. You can also see that the definition of a function uses vector literals to denote the arguments to a function and for the `(let)` form.

The third function is more complex. The function `(list-maker-fun)` takes two arguments: first a vector of values to operate on, which is called `x`, and second, a function (called `f`). If we were to write it in Java, it might look a bit like this:

```
public List<Object> listMakerFun(List<Object> x,
                                 Function<Object, Object> f) {
  return x.stream()
        .map(o -> List.of(o, f.apply(o)))
        .collect(toList());
}
```

The role of the inline anonymous function in Clojure is played by the lambda expression in the Java code. However, it is important not to overstate the equivalence of these two code listings—Clojure and Java are *very* different languages.

**NOTE** Functions that take other functions as arguments are called higher-order functions. We'll meet them properly in chapter 15.

Let's take a look at how `(list-maker-fun)` works.

Listing 10.2 Working with functions

```
user=> (list-maker-fun ["a"] const-fun1)
(("a" 1))

user=> (list-maker-fun ["a" "b"] const-fun1)
(("a" 1) ("b" 1))
```

```
user=> (list-maker-fun [2 1 3] ident-fun)
((2 2) (1 1) (3 3))

user=> (list-maker-fun [2 1 3] "a")
java.lang.ClassCastException: java.lang.String cannot be cast to
  clojure.lang.IFn
```

Note that when you're typing these expressions into the REPL, you're interacting with the Clojure compiler. The expression `(list-maker-fun [2 1 3] "a")` fails to run (although it does compile) because `(list-maker-fun)` expects its second argument to be a function, which a string isn't. So although the Clojure compiler outputs bytecode for the form, it fails with a runtime exception.

**NOTE** In Java, we can write valid code like `Integer.parseInt("foo")`, which will compile fine but will always fail at runtime. The Clojure situation is similar.

This example shows that when interacting with the REPL, you still have a certain amount of static typing in play because Clojure isn't an interpreted language. Even in the REPL, every Clojure form that is typed is compiled to JVM bytecode and linked into the running system. The Clojure function is compiled to JVM bytecode when it's defined, so the `ClassCastException` occurs because of a static typing violation in the JVM.

Listing 10.3 shows a longer piece of Clojure code, the *Schwartzian transform*. This is a piece of programming history, made popular by the Perl programming language in the 1990s. The idea is to do a sort operation on a vector, based not on the provided vector but on some property of the elements of the vector. The property values to sort on are found by calling a *keying function* on the elements.

The definition of the Schwartzian transform in listing 10.3 calls the keying function `key-fn`. When you actually want to call the `(schwartz)` function, you need to supply a function to use for keying. In this code sample, we use our old friend, `(ident-fun)`, from listing 10.1.

Listing 10.3 Schwartzian transform

```
user=> (defn schwartz [x key-fn]
  (map (fn [y] (nth y 0))                ❶
    (sort-by (fn [t] (nth t 1))          ❷
      (map (fn [z] (let [w z]            ❸
        (list w (key-fn w))
```

```
    )) x))))
#'user/schwartz

user=> (schwartz [2 3 1 5 4] ident-fun)
(1 2 3 4 5)

user=> (apply schwartz [[2 3 1 5 4] ident-fun])
(1 2 3 4 5)
```

1. Makes a list consisting of pairs using the keying function
2. Sorts the pairs based on the values of the keying function
3. Constructs a new list by reducing—taking only the original value from each pair

This code is performing three separate steps, which may seem a little inside out at first glance. The steps are shown in figure 10.5.

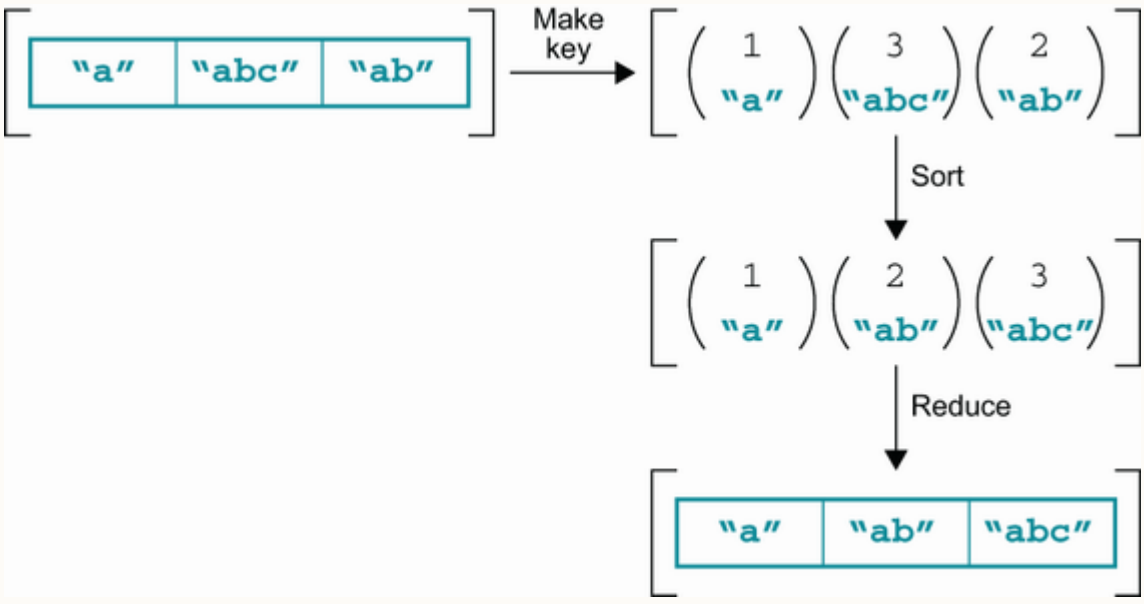



Figure 10.5 The Schwartzian transform

Note that in listing 10.3 we introduced a new form: `(sort-by)`. This is a function that takes two arguments: a function to do the sorting and a vector to be sorted. We've also showcased the `(apply)` form, which takes two arguments: a function to call and a vector of arguments to pass to it.

One amusing aspect of the Schwartzian transform is that the person for whom it was named was deliberately aping Lisp when he came up with the Perl version. Representing it in the Clojure code here means we've come full circle—back to a Lisp again!

The Schwartzian transform is a useful example that we'll refer back to later. It contains just enough complexity to demonstrate quite a few useful concepts. Now let's move on to discuss loops in Clojure, which work a bit differently from what you may be used to.

### 0.2.5 Loops in Clojure

Loops in Java are a fairly straightforward proposition: the developer can choose from a `for`, a `while`, and a couple of other loop types. Usually central is the concept of repeating a group of statements until a condition (often expressed in terms of a mutable variable) is met.

This presents us with a slight conundrum in Clojure: how can we express a `for` loop when there are no mutable variables to act as the loop index? In more traditional Lisps, this situation is often solved by rewriting iterative loops into a form that uses recursion.

However, the JVM doesn't guarantee optimizing tail recursion (as is required by Scheme and other Lisps), so naïvely using recursion can cause the stack to blow up. We will have more to say about this issue in chapter 15.

Instead, Clojure provides some useful constructions to allow looping without increasing the size of the stack. One of the most common is `loop-recur`. The next snippet shows how `loop-recur` can be used to build up a simple construction similar to a Java for loop:

```
(defn like-for [counter]
  (loop [ctr counter]      ❶
    (println ctr)
    (if (< ctr 10)
      (recur (inc ctr))    ❷
      ctr
  )))
```

❶ The loop entry point

❷ The recur point where we jump backward

The `(loop)` form takes a vector of arguments of local names for symbols—effectively aliases as `(let)` does. Then, when execution reaches the `(recur)` form (which it will do in this example only if the `ctr` alias is less than 10), the `(recur)` causes control to branch back to the `(loop)` form but with the new value specified. This is similar to a rather primitive form of Java loop construction, shown here:

```
public int likeFor(int ctr) {
      LOOP: while (true) {
         System.out.println(ctr);
         if (ctr < 10) {
            ctr = ctr + 1;
            continue LOOP;
         } else {
            return ctr;
         }
      }
   }
```

However, for a functional programmer, the only common reason to return early is if some condition is met. However, functions return the result of the last form evaluated, and `(if)` basically already does this for us.

In our example, we put the `(recur)` in the body of the `if` and the countervalue in the `else` position. This allows us to build up iteration-style constructs (such as the equivalent of Java's `for` and `while` loops) but still have a functional flavor to the implementation. We'll now turn to our next topic, which is a look at useful shorthand in Clojure syntax to help make your programs even shorter and less verbose.

### 10.2.6 Reader macros and dispatch

Clojure has syntax features that surprise many Java programmers. One of them is the lack of operators. This has the side effect of relaxing Java's restrictions on which characters can be used in function names. You've already met functions such as `(identical?)`, which would be illegal in Java, but we haven't addressed the issue of exactly which characters are and aren't allowed in symbols.

Table 10.2 lists the characters that aren't allowed in Clojure symbols. These are all characters that are reserved by the Clojure parser for its own use. They're usually referred to as *reader macros*, and they are effectively a special character sequence, which, when seen by the reader (the first part of the Clojure compiler), modifies the reader's behavior.

For example, the `;` reader macro is how Clojure implements single-line comments. When the reader sees `;`, it immediately ignores all remaining characters on this line, then resets to take the next line of input.

**NOTE** Later we will meet Clojure's general (or regular) macros. It is important not to confuse a reader macro with a regular macro.

Reader macros exist only for syntactical concision and convenience, not to provide a full general-purpose metaprogramming capability.

Table 10.2 Reader macros

| Character | Name | Meaning |
|---|---|---|
| `'` | Quote | Expands to `(quote)`; yields the unevaluated form |
| `;` | Comment | Marks a comment to end of line; like `//` in Java |
| `\` | Character | Produces a literal character, for example, `\n` for newline |
| `@` | Deref | Expands to `(deref)`, which takes in a var object and returns the value in that object (the opposite action of the `(var)` form); has additional meaning in a transactional memory context (see chapter 15) |
| `^` | Metadata | Attaches a map of metadata to an object; see the Clojure documentation for details |
| `` ` `` | Syntax-quote | Form of quote often used in macro definitions; see the macros section for details |
| `#` | Dispatch | Has several different subforms; see table 10.3 |

The dispatch reader macro has several different subforms, depending on what follows the `#` character. Table 10.3 shows the different possible forms.

Table 10.3 The subforms of the dispatch reader macro

| Dispatch form | Meaning |
|---|---|
| `#'` | Expands to (var) |
| #{} | Creates a set literal, as discussed in section 10.2.2 |
| `#()` | Creates an anonymous function literal; useful for single uses where `(fn)` is too wordy |
| #_ | Skips the next form; can be used to produce a multiline comment, via #_( ... multiline ...) |
| `#"<pattern>"` | Creates a regular expression literal (as a `java.util.regex.Pattern` object) |

A couple of additional points follow from the dispatch forms. The var-quote ( `#'` ) form, shown next, explains why the REPL behaves as it does after a `(def)` :

```
user=> (def someSymbol)

#'user/someSymbol
```

The `(def)` form returns the newly created var object named `someSymbol` , which lives in the current namespace (which is `user` in the REPL), so `#'user/someSymbol` is the full value of what's returned from `(def)` .

The anonymous function literal `#()` also has a major innovation to reduce verboseness—it omits the vector of arguments and instead uses a special syntax to allow the Clojure reader to infer how many arguments are required for the function literal. The syntax is `%N` , where N is the number of the argument to the function.

Let's return to an earlier example and see how to use it with anonymous functions. Recall the `(list-maker-fun)` that takes two arguments (a list and a function) and creates a new list by applying the function to each element in turn:

```
(defn list-maker-fun [x f]
  (map (fn [z] (let [w z]
```

```
    (list w (f w))
  )) x))
```

Rather than going to all the bother of defining a separate symbol, we can call this function with an inline function as follows:

```
user=> (list-maker-fun ["a" "b"] (fn [x] x))
(("a" "a") ("b" "b"))
```

But we can go one step further and use the move compact `#()` syntax like this:

```
user=> (list-maker-fun ["a" "b"] #(do %1))
(("a" "a") ("b" "b"))
```

This example is a little unusual, because we're using the `(do)` form we met back in the table of basic special forms, but it works. Now, let's simplify `(list-maker-fun)` itself using the `#()` form:

```
(defn list-maker-fun [x f]
  (map #(list %1 (f %1)) x))
```

The Schwartzian transform also makes an excellent use case to see how to use this syntax in a more complex example, as shown in the next code sample.

Listing 10.4 Rewritten Schwartzian transform

```
(defn schwartz [x key-fn]
  (map #(nth %1 0)          ❶
    (sort-by #(nth %1 1)    ❶
      (map #(let [w %1]     ❶
        (list w (key-fn w))
      ) x))))
```

❶ Anonymous function literals corresponding to the three steps

The use of %1 as a placeholder for a function literal's argument (and %2, %3, and so on for subsequent arguments) makes the usage really stand out and makes the code a lot

easier to read. This visual clue can be a real help for the programmer, similar to the arrow symbol used in lambda expressions in Java.

As you've seen, Clojure relies heavily on the concept of functions as the basic unit of computation, rather than on objects, which are the staple of languages like Java. The natural setting for this approach is functional programming, which is our next topic.

## 10.3 Functional programming and closures

We're now going to turn to the scary world of functional programming in Clojure. Or rather, we're *not*, because it's just not that scary. In fact, we've been doing functional programming for this entire chapter; we just didn't tell you to not put you off.

As we mentioned in section 8.1.3, functional programming is a somewhat nebulous concept—all it can be relied upon to mean is that a function is a value. A function can be passed around, placed in variables and manipulated, just like `2` or `"hello."` But so what? We did that back in our very first example: `(def hello (fn [] "Hello world"))`. We created a function (one that takes no arguments and returns the string `"Hello world"`) and bound it to the symbol `hello`. The function was just a value, not fundamentally different for a value like `2`.

In listing 10.3, we introduced the Schwartzian transform as an example of a function that takes another function as an input value. Again, this is just a function taking a particular type as one of its input arguments. The only thing that's slightly different about it is that the type it's taking is a function.

It's probably also a good time to introduce the `(filter)` form, shown next, which should remind you of the similarly named method in Java Streams:

```
user=> (defn gt4 [x] (> x 4))
#'user/gt4
user=> (filter gt4 [1 2 3 4 5 6])
(5 6)
```

There is also the `(reduce)` form, to complete the set of filter-map-reduce operations. It is most commonly seen in two variants, one that takes an initial starting value (sometimes called a "zero") and one that doesn't:

```
user=> (reduce + 1 [2 3 4 5])
15
```

```
user=> (reduce + [1 2 3 4 5])
15
```

What about closures? Surely they're really scary, right? Well, not so much. Let's take a look at a simple example that should hopefully remind you of some of the examples we did for Kotlin in chapter 9:

```
user=> (defn adder [constToAdd] #(+ constToAdd %1))
#'user/adder

user=> (def plus2 (adder 2))
#'user/plus2

user=> (plus2 3)
5

user=> 1:9 user=> (plus2 5)
7
```

You first set up a function called `(adder)`. This is a function that makes other functions. If you're familiar with the Factory Method pattern in Java, you can think of this as kind of a Clojure equivalent. There's nothing strange about functions that have other functions as their return values—this is a key part of the concept that functions are just ordinary values.

Notice that this example uses the shorthand form `#()` for an anonymous function literal. The function `(adder)` takes in a number and returns a function, and the function returned from `(adder)` takes one argument.

You then use `(adder)` to define a new form: `(plus2)`. This is a function that takes one numeric argument and adds `2` to it. The value that was bound to `constToAdd` inside `(adder)` was `2`. Now let's make a new function:

```
user=> (def plus3 (adder 3))
#'user/plus3

user=> (plus3 4)
7

user=> (plus2 4)
6
```

This shows that you can make a different function, `(plus3)`, that has a different value bound to `constToAdd`. We say that the functions `(plus3)` and `(plus2)` have *captured*, or *closed over* a value from their environment. Note that the values that were captured by `(plus3)` and `(plus2)` were different and that defining `(plus3)` had no effect on the value captured by `(plus2)`.

Functions that close over some values in their environment are called *closures*; `(plus2)` and `(plus3)` are examples of closures. The pattern whereby a function-making function returns another, simpler function that has closed over something is a very common one in languages that have closures.

**NOTE** Remember that although Clojure will compile any syntactically valid form, the program will throw a runtime exception if a function is called with the wrong number of arguments. A two-argument function could not be used in a place where a one-argument function was expected.

We will have a lot more to say about functional programming in context in chapter 15. Now let's turn to a powerful Clojure feature— sequences.

## 10.4 Introducing Clojure sequences

Clojure has a powerful core abstraction called the *sequence* or, more usually, *seq*.

**NOTE** Sequences are a major part of writing Clojure code that utilizes the strengths of the language, and they'll provide an interesting contrast to how Java handles similar concepts.

The seq type roughly corresponds to collections and iterators in Java, but seqs have somewhat different properties. The fundamental idea is that seqs essentially merge some of the features of both Java types into one concept. This is motivated by wanting the three following things:

- Immutability, allowing the seqs to be passed around between functions (and threads) without problems
- A more robust iterator-like abstraction, especially for multipass algorithms
- The possibility of *lazy sequences* (more on these later)

Of these three things, the one that Java programmers sometimes struggle with the most is the immutability. The Java concept of an iterator is inherently mutable, partly because it does not provide a cleanly separable interface. In fact, Java's `Iterator` violates the

Single Responsibility Principle because `next()` does the following *two* things logically distinct things when called:

- It returns the currently pointed-at element.
- It mutates the iterator by advancing the element pointer.

The seq is based on functional ideas and avoids the mutation by dividing up the capabilities of `hasNext()` and `next()` in a different way. Let's meet a slightly simplified version of another of Clojure's most important interfaces, `clojure.lang.ISeq`:

```
interface ISeq {
   Object first();     (1)
   ISeq rest();        (2)
}
```

(1) Returns the object that is first in the seq

(2) Returns a new seq that contains all the elements of the old seq, except the first

Now, the seq is never mutated. Instead a new seq value is created every time we call `rest()`, which is when we would have stepped the iterator to the next value. Let's look at some code to show how we might implement this in Java:

```
public class ArraySeq implements ISeq {
   private final int index;                    (1)
   private final Object[] values;              (1)

   private ArraySeq(int index, Object[] values) {
      this.index = index;
      this.values = values;
   }

   public static ArraySeq of(List<Object> objects) {   (2)
      if (objects == null || objects.size() == 0) {
         return Empty.of();
      }
      return new ArraySeq(0, objects.toArray());
   }

   @Override
   public Object first() {
      return values[index];
```

```
    }

    @Override
    public ISeq rest() {
      if (index >= values.length - 1) {
        return Empty.of();                    ❸
      }
      return new ArraySeq(index + 1, values);
    }

    public int count() {
      return values.length - index;
    }
  }
```

❶ Final fields

❷ Factory method that takes a List

❸ Needs an empty implementation as well

As you can see, we need a special-case seq for the end of the sequence. Let's represent it as an inner class within `ArraySeq` like this:

```
    public static class Empty extends ArraySeq {
      private static Empty EMPTY = new Empty(-1, null);

      private Empty(int index, Object[] values) {
        super(index, values);
      }

      public static Empty of() {
        return EMPTY;
      }

      @Override
      public Object first() {
        return null;
      }

      @Override
      public ISeq rest() {
        return of();
      }
```

```
    public int count() {
      return 0;
    }
  }
```

Let's see this in action:

```
ISeq seq = ArraySeq.of(List.of(10000,20000,30000));
var o1 = seq.first();
var o2 = seq.first();
System.out.println(o1 == o2);
```

As expected, calls to `first()` are *idempotent*—they do not change the seq and will repeatedly return the same value.

Let's look at how we'd write a loop in Java using `ISeq`:

```
while (seq.first() != null) {
  System.out.println(seq.first());
  seq = seq.rest();
}
```

This example shows how we deal with one objection that some Java programmers sometimes have with the immutable seq approach: "What about all the garbage?"

It's true that each call to `rest()` will create a new seq, which is an object. However, if you look closely at the implementing code you can see that we're careful not to duplicate `values`—the array storage. Copying that would be expensive, so we don't do that.

All we're really creating at each step is a tiny object that contains an int and a reference to an object. If these temporaries aren't stored anywhere, they'll fall out of scope as we walk down the seq and very quickly become eligible for garbage collection.

**NOTE** The method bodies for `Empty` do not refer to either `index` or `values`, so we are free to use special values (-1 and `null`), which would not be able to be reached by any other instance of `ArraySeq`—this is a debugging aid.

Let's switch back into Clojure now that we've explained some of the theory of seqs using Java.

**NOTE** The real `ISeq` interface that all Clojure sequences implement is a little more complex than the version we've met so far, but the basic intent is the same.

Some core functions that relate to sequences are shown in table 10.4. Note that none of these functions will mutate their input arguments; if they need to return a different value, it will be a different seq.

Table 10.4 Basic sequence functions

| Function | Effect |
|---|---|
| `(seq <coll>)` | Returns a seq that acts as a "view" onto the collection acted upon |
| (first <coll>) | Returns the first element of the collection, calling ( seq ) on it first if necessary; returns nil if the collection is nil |
| `(rest <coll>)` | Returns a new seq, made from the collection, minus the first element |
| (seq? <o>) | Returns true if o is a seq (meaning, if it implements ISeq ) |
| `(cons <elt> <coll>)` | Returns a seq made from the collection, with the additional element prepended |
| (conj <coll> <elt>) | Returns a new collection with the new element added to the appropriate end—the end for vectors and the head for lists |
| `(every? <pred-fn> <coll>)` | Returns true if `(pred-fn)` returns logical-true for every item in the collection |

Clojure differs from other Lisps because `(cons)` requires the second argument to be a collection (or, really an `ISeq`). In general, a lot of Clojure programmers favor `(conj)` over `(cons)`. Here are a few examples:

```
user=> (rest '(1 2 3))
(2 3)

user=> (first '(1 2 3))
1
```

```
user=> (rest [1 2 3])
(2 3)

user=> (seq ())
nil

user=> (seq [])
nil

user=> (cons 1 [2 3])
(1 2 3)

user=> (every? is-prime [2 3 5 7 11])
true
```

One important point to note is that Clojure lists are their own seqs, but vectors aren't. In theory, you shouldn't be able to call `(rest)` on a vector. The reason you're able to is that `(rest)` acts by calling `(seq)` on the vector before operating on it.

**NOTE** Many of the sequence functions take more general objects than seqs and will call `(seq)` on them before they begin.

In the next section, we're going to explore some of the basic properties and uses of the seq abstraction, paying special attention to variadic functions. Later, in chapter 15, we'll meet lazy sequences—a very important functional technique.

### 10.4.1 Sequences and variable-arity functions

We've delayed discussing fully one powerful feature of Clojure's approach to functions until now. This is the natural ability to easily have variable numbers of arguments to functions, sometimes called the *arity* of functions. Functions that accept variable numbers of parameters are called *variadic*, and they are frequently used when operating on seqs.

**NOTE** Java supports variadic methods, with a syntax in which the final parameter of a method is shown with `...` on the type, to indicate that any number of parameters of that type are allowed at the end of the parameter list.

As a trivial example, consider the constant function `(const-fun1)` that we discussed in listing 10.1. This function takes in a single argument and discards it, always returning the

value 1. But consider what happens when you pass more than one argument to `(const-fun1)` like this:

```
user=> (const-fun1 2 3)
java.lang.IllegalArgumentException:
  Wrong number of args (2) passed to: user$const-fun1 (repl-1:32)
```

The Clojure compiler cannot enforce compile-time static checks on the number (and types) of arguments passed to `(const-fun1)`, and instead we have to risk runtime exceptions.

This seems overly restrictive, especially for a function that simply discards all of its arguments and returns a constant value. What would a function that could take any number of arguments look like in Clojure?

The following listing shows how to do this for a version of the `(const-fun1)` constant function from earlier in the chapter. We've called it `(const-fun-arity1)`, for *constant function 1* with variable *arity*.

**NOTE** This is, in fact, a homebrew version of the `(constantly)` function provided in the Clojure standard function library.

Listing 10.5 Variable arity function

```
user=> (defn const-fun-arity1
  ([] 1)                  ❶
  ([x] 1)                 ❶
  ([x & more] 1)          ❶
)
#'user/const-fun-arity1

user=> (const-fun-arity1)
1

user=> (const-fun-arity1 2)
1

user=> (const-fun-arity1 2 3 4)
1
```

❶ Multiple function definitions with different signatures

The key is that the function definition is followed not by a vector of function parameters and then a form defining the behavior of the function. Instead, there is a list of pairs, with each pair consisting of a vector of parameters (effectively the signature of this version of the function) and the implementation for this version of the function.

This can be thought of as a similar concept to method overloading in Java. Alternatively, it could also be seen as related to pattern matching (which we met in chapter 3). However, because Clojure is a dynamically typed language, there is no equivalent of type patterns, and so the connection is not as strong as it might be.

The usual convention is to define a few special-case forms (that take none, one, or two parameters) and an additional form that has as its last parameter a seq. In listing 10.5, this is the form that has the parameter vector of `[x & more]`. The `&` sign indicates that this is the variadic version of the function.

Sequences are a powerful Clojure innovation. In fact, a large part of learning to think in Clojure is to start thinking about how the seq abstraction can be put to use to solve your specific coding problems. Another important innovation in Clojure is the integration between Clojure and Java, which is the subject of the next section.

## 10.5 Interoperating between Clojure and Java

Clojure was designed from the ground up to be a JVM language and to not attempt to completely hide the JVM character from the programmer. These specific design choices are apparent in a number of places. For example, at the type-system level, Clojure's lists and vectors both implement `List`—the standard interface from the Java collections library. In addition, it's very easy to use Java libraries from Clojure and vice versa. These properties are extremely useful, because Clojure programmers can make use of the rich variety of Java libraries and tooling, as well as the performance and other features of the JVM.

In this section, we'll cover a number of aspects of this interoperability decision, specifically:

- Calling Java from Clojure
- How Java sees the type of Clojure functions
- Clojure proxies
- Exploratory programming with the REPL
- Calling Clojure from Java

Let's start exploring this integration by looking at how to access Java methods from Clojure.

### 10.5.1 Calling Java from Clojure

Consider this piece of Clojure code being evaluated in the REPL:

```
user=> (defn lenStr [y] (.length (.toString y)))
#'user/lenStr

user=> (schwartz ["bab" "aa" "dgfwg" "droopy"] lenStr)
("aa" "bab" "dgfwg" "droopy")
```

In this snippet, we've used the Schwartzian transform to sort a vector of strings by their lengths. To do that, we've used the forms `(.toString)` and `(.length)`, which are Java methods. They're being called on the Clojure objects. The period at the start of the symbol means that the runtime should invoke the named method on the next argument. This is achieved by the behind-the-scenes use of another macro that we haven't met yet— `(.)`.

Recall that all Clojure values defined by `(def)` or a variant of it are placed into instances of `clojure.lang.Var`, which can house any `java.lang.Object`, so any method that can be called on `java.lang.Object` can be called on a Clojure value. Some of the other forms for interacting with the Java world are

```
(System/getProperty "java.vm.version")
```

for calling static methods (in this case the `System.getProperty()` method) and

```
Boolean/TRUE
```

for accessing static public variables (such as constants).

The familiar "Hello World" example looks like this:

```
user=> (.println System/out "Hello World")
Hello World
nil
```

Note that the final `nil` is because, of course, all Clojure forms must return a value, even if they are a call to a `void` Java method.

In these three examples, we've implicitly used Clojure's namespaces concept, which is similar to Java packages and has mappings from shorthand forms to Java package names for common cases, such as the preceding ones.

### 0.5.2 The nature of Clojure calls

A function call in Clojure is compiled to a JVM method call. The JVM does not guarantee optimizing away tail recursion, which Lisps (especially Scheme implementations) usually do. Some other Lisp dialects on the JVM take the viewpoint that they want true tail recursion, so they are prepared to have a Lisp function call not be exactly equivalent to a JVM method call under all circumstances. Clojure, however, fully embraces the JVM as a platform, even at the expense of full compliance with usual Lisp practice.

If you want to create a new instance of a Java object and manipulate it in Clojure, you can easily do so by using the `(new)` form. This has an alternative short form, which is the class name followed by the full stop, which boils down to another use of the `(.)` macro, as shown next:

```
(import '(java.util.concurrent CountDownLatch LinkedBlockingQueue))

(def cdl (new CountDownLatch 2))

(def lbq (LinkedBlockingQueue.))
```

Here we're also using the `(import)` form, which allows multiple Java classes from a single package to be imported in just one line.

We mentioned earlier that there's a certain amount of alignment between Clojure's type system and that of Java. Let's take a look at this concept in a bit more detail.

### 0.5.3 The Java type of Clojure values

From the REPL, it's very easy to take a look at the Java types of some Clojure values as follows:

```
user=> (.getClass "foo")
java.lang.String
```

```
user=> (.getClass 2.3)
java.lang.Double

user=> (.getClass [1 2 3])
clojure.lang.PersistentVector

user=> (.getClass '(1 2 3))
clojure.lang.PersistentList

user=> (.getClass (fn [] "Hello world!"))
user$eval110$fn__111
```

The first thing to notice is that all Clojure values are objects; the primitive types of the JVM aren't exposed by default (although there are ways of getting at the primitive types for the performance-conscious). As you might expect, the string and numeric values map directly onto the corresponding Java reference types ( `java.lang.String` , `java.lang.Double` , and so on).

The anonymous "Hello world!" function has a name that indicates that it's an instance of a dynamically generated class. This class will implement the interface `IFn` , which is the very important interface that Clojure uses to indicate that a value is a function.

As we discussed a bit earlier, seqs implement the `ISeq` interface. They will typically be one of the concrete subclasses of the abstract `ASeq` or the lazy implementation, `LazySeq` (we'll meet laziness in chapter 15 when we talk about advanced functional programming).

We've looked at the types of various values, but what about the storage for those values? As we mentioned at the start of this chapter, `(def)` binds a symbol to a value and, in doing so, creates a var. These vars are objects of type `clojure.lang.Var` (which implements `IFn` , among other interfaces).

### 10.5.4 Using Clojure proxies

Clojure has a powerful macro called `(proxy)` that enables you to create a bona fide Clojure object that extends a Java class (or implements an interface). For example, the next listing revisits an earlier example (using the `ScheduledThreadPoolExecutor` from chapter 6), but the heart of the execution example is now done in a fraction of the code, due to Clojure's more compact syntax.

Listing 10.6 Revisiting scheduled executors

```
(import '(java.util.concurrent Executors LinkedBlockingQueue TimeUnit))

(def stpe (Executors/newScheduledThreadPool 2))          ❶

(def lbq (LinkedBlockingQueue.))

(def msgRdr (proxy [Runnable] []                         ❷
  (run [] (.println System/out (.toString (.poll lbq))))
))

(def rdrHndl
  (.scheduleAtFixedRate stpe msgRdr 10 10 TimeUnit/MILLISECONDS))
```

❶ Factory method to create an executor

❷ Defines an anonymous implementation of Runnable

The general form of `(proxy)` follows:

```
(proxy [<superclass/interfaces>] [<args>] <impls of named functions>+)
```

The first vector argument holds the interfaces that this proxy class should implement. If the proxy should also extend a Java class (and it can, of course, extend only one Java class), that class name must be the first element of the vector.

The second vector argument comprises the parameters to be passed to a superclass constructor. This is quite often the empty vector, and it will certainly be empty for all cases where the `(proxy)` form is just implementing Java interfaces.

After these two arguments come the forms that represent the implementations of individual methods, as required by the interfaces or superclasses specified. In our example, the proxy needs to implement only `Runnable`, so that is the only symbol in the first vector of arguments. No superclass parameters are needed, so the second vector is empty (as it very often is).

Following the two vectors, comes a list of forms that define the methods that the proxy will implement. In our case, that is just `run()`, and we give it the definition `(run [] (.println System/out (.toString (.poll lbq))))`. This is, of course, just the Clojure way of writing this bit of Java:

```
public void run() {
   System.out.println(lbq.poll().toString());
}
```

The `(proxy)` form allows for the simple implementation of any Java interface. This leads to an intriguing possibility—that of using the Clojure REPL as an extended playpen for experimenting with Java and JVM code.

### 10.5.5 Exploratory programming with the REPL

The key concept of exploratory programming is that with less code to write, due to Clojure's syntax, and the live, interactive environment that the REPL provides, the REPL can be a great environment for not only exploring Clojure programming but for learning about Java libraries as well.

Let's consider the Java list implementations. They have an `iterator()` method that returns an object of type Iterator. But Iterator is an interface, so you might be curious about what the real implementing type is. Using the REPL, it's easy to find out as shown here:

```
user=> (import '(java.util ArrayList LinkedList))
java.util.LinkedList

user=> (.getClass (.iterator (ArrayList.)))
java.util.ArrayList$Itr

user=> (.getClass (.iterator (LinkedList.)))
java.util.LinkedList$ListItr
```

The `(import)` form brings in two different classes from the `java.util` package. Then you can use the `getClass()` Java method from within the REPL just as you did in section 10.5.3. As you can see, the iterators are actually provided by inner classes. This perhaps shouldn't be surprising; as we discussed in section 10.4, iterators are tightly bound up with the collections they come from, so they may need to see internal implementation details of those collections.

Notice that in the preceding example, we didn't use a single Clojure construct—just a little bit of syntax. Everything we were manipulating was a true Java construct. Let's suppose, though, that you wanted to use a different approach and use the powerful abstrac-

tions that Clojure brings within a Java program. The next subsection will show you just how to accomplish this.

### 10.5.6 Using Clojure from Java

Recall that Clojure's type system is closely aligned with Java's. The Clojure data structures are all true Java collections that implement the whole of the mandatory part of the Java interfaces. The optional parts aren't usually implemented, because they're often about mutation of the data structures, which Clojure doesn't support.

This alignment of type systems opens the possibility of using Clojure data structures in a Java program. This is made even more viable by the nature of Clojure itself—it's a compiled language with a calling mechanism that matches that of the JVM. This minimizes the runtime aspects and means a class obtained from Clojure can be treated almost like any other Java class. Interpreted languages would find it a lot harder to interoperate and would typically require a minimal non-Java language runtime for support.

The next example shows how Clojure's seq construct can be used on an ordinary Java string. For this code to run, `clojure.jar` will need to be on the classpath:

```
ISeq seq = StringSeq.create("foobar");

while (seq != null) {
  Object first = seq.first();
  System.out.println("Seq: "+ seq +" ; first: "+ first);
  seq = seq.next();
}
```

The preceding code snippet uses the factory method `create()` from the `StringSeq` class. This provides a seq view on the character sequence of the string. The `first()` and `next()` methods return new values, as opposed to mutating the existing seq, just as we discussed in section 10.4.

In the next section, we'll move on to talk Clojure's macros. This is a powerful technique that allows the experienced programmer to effectively modify the Clojure language itself. This capability is common in languages like Lisp but rather alien to Java programmers, so it warrants an entire section to itself.

## 10.6 Macros

In chapter 8, we discussed the rigidity of the language grammar of Java. By contrast, Clojure provides and actively encourages macros as a mechanism to provide a much more flexible approach, allowing the programmer to write more or less ordinary program code that behaves in the same way as built-in language syntax.

**NOTE** Many languages have macros (including C++), and they mostly all operate in a roughly similar way—by providing a special phase of source code compilation, often the very first phase.

For example, in the C language, the first step is *preprocessing*, which removes comments, inlines included files, and expands macros, which are the different types of *preprocessor directives* such as `#include` and `#define`.

However, although C macros were very powerful, they also make it possible for engineers to produce some very subtly confusing code that is hard to understand and debug. To avoid this complexity, the Java language never implemented a macro system or a preprocessor.

C macros work by providing very simple text-replacement capabilities during the preprocessing phase. Clojure macros are safer, because they work within the syntax of Clojure itself. Effectively, they allow the programmer to create a special kind of function that is evaluated (in a special way) at compile time. The macro can transform source code during compilation during what is referred to as *macro expansion time*.

**NOTE** The key to the power of macros is the fact that Clojure code is written down as a valid Clojure data structure—specifically as a list of forms.

We say that Clojure, like other Lisps (and a few other languages), is *homoiconic*, which means that programs are represented in the same way as data. Other programming languages, like Java, write their source code as a string, and without parsing that string in a Java compiler, the structure of the program cannot be determined.

Recall that Clojure compiles source code as it is encountered. Many Lisps are interpreted languages, but Clojure is not. Instead, when Clojure source code is loaded, it is compiled on the fly into JVM bytecode. This can give the superficial impression that Clojure is interpreted, but the (very simple) Clojure compiler is hiding just below the surface.

**NOTE** A Clojure form is a list, and a macro is essentially a function that does not evaluate its arguments but instead manipulates them to return another list, which will then be

compiled as a Clojure form.

To demonstrate this, let's try to write a macro form that acts like the opposite of `(if)`. In some languages, this would be represented with the `unless` keyword, so in Clojure it will be an `(unless)` form. What we want is a form that looks like `(if)` but behaves as the logical opposite, like this:

```
user=> (def test-me false)
#'user/test-me

user=> (unless test-me "yes")
"yes"

user=> (def test-me true)
#'user/test-me

user=> (unless test-me "yes")
nil
```

Note that we don't provide the equivalent of an `else` condition. This somewhat simplifies the example and "unless ... else" sounds weird anyway. In our examples, if the `unless` logical test fails, the form evaluates to `nil`.

If we try to write this using `(defn)`, we can write a simple first attempt like this (spoiler: it won't actually work properly):

```
user=> (defn unless [p t]
  (if (not p) t))
#'user/unless

user=> (def test-me false)
#'user/test-me

user=> (unless test-me "yes")
"yes"

user=> (def test-me true)
#'user/test-me

user=> (unless test-me "yes")
nil
```

This seems fine. However, consider that we want `(unless)` to work the same way as `(if)` —in particular, the `then` form should evaluated only if the Boolean predicate condition is true. In other words, for `(if)` we see this behavior:

```
user=> (def test-me true)
#'user/test-me

user=> (if test-me (do (println "Test passed") true))
Test passed
true

user=> (def test-me false)
#'user/test-me

user=> (if test-me (do (println "Test passed") true))
nil
```

When we try to use our `(unless)` function in the same way, the problem becomes clear, as illustrated here:

```
user=> (def test-me false)
#'user/test-me

user=> (unless test-me (do (println "Test passed") true))
Test passed
true

user=> (def test-me true)
#'user/test-me

user=> (unless test-me (do (println "Test passed") true))
Test passed
nil
```

Regardless of whether the predicate is true or false, the `then` form is still evaluated, and as it is `(println)` in our example, it produces output, which provides the clue that lets us know that the evaluation is taking place. To solve this problem, we need to handle the forms that we are passed *without evaluating them*. This is essentially a (slightly different) kind of the laziness concept that is so important in functional programming (and which we will describe in detail in chapter 15). The special form `(defmacro)` is used to declare a new macro, like this:

```
(defmacro unless [p t]
  (list 'if (list 'not p) t))
```

Let's see if it does the right thing:

```
user=> (def test-me true)
#'user/test-me

user=> (unless test-me (do (println "Test passed") true))
nil

user=> (def test-me false)
#'user/test-me

user=> (unless test-me (do (println "Test passed") true))
Test passed
true
```

This now behaves as we want it to: essentially, the `(unless)` form now looks and behaves just like the built-in `(if)` special form.

As you can see, one of the drawbacks of writing macros is that a lot of quoting is involved. The macro transforms its arguments to a new Clojure form at compile time, so it is natural that the output should be a `(list)`.

The list contains Clojure symbols that will be evaluated at runtime, so anything that we do not explicitly need to evaluate during macro expansion must be quoted. This relies upon the fact that macros receive their arguments at compile time, so they are available as unevaluated data.

In our example, we need to quote everything that is *not* one of our arguments—these will be string-replaced as symbols during expansion. This gets pretty cumbersome fairly quickly. Can we do better?

Let's meet a helpful tool that might point us in the right direction. When writing or debugging macros, the `(macroexpand-1)` form can be very useful. If this form is passed a macro form, it expands the macro and returns the expansion. If the passed form is not a macro, it just returns the form, for example:

```
user=> (macroexpand-1 '(unless test-me (do (println "Test passed") true)))
(if (not test-me) (do (println "Test passed") true))
```

What we would really like is the ability to write macros that look like their macro-expanded form without the huge amount of quoting that we've seen in examples so far.

**NOTE** Full macro expansion, using the form `(macroexpand)`, is then constructed by just repeatedly calling the former, simpler form. When applying `(macroexpand-1)` is a no-op, macro expansion is over. The key to this capability is the special reader macro `` ` ``, which is pronounced "syntax-quote" and which we previewed earlier in the chapter as part of the section about reader macros. The syntax quoting reader macro works by basically quoting everything in the following form. If you want something to *not* be quoted, you have to use the syntax-unquote ( `~` ) operator to exempt a value from syntax quoting. This means our example macro `(unless)` can be written as follows:

```
(defmacro unless [p t]
  `(if (not ~p) ~t))
```

This form is now much clearer and closer to the form we see when macro expanding. The `~` character provides a nice visual clue to let us know that those symbols will be replaced when the macro is expanded. This fits nicely with the idea of a macro as a compile-time code template.

Along with syntax-quote and -unquote, some important special variables are sometimes used in macro definitions. Of these, two of the most common follow:

- `&form` —the expression that is being invoked
- `&env` —a map of local bindings at the point of macro expansion

Full details of the information that can be obtained from each special variable can be found in the Clojure documentation.

We should also note that care needs to be taken when writing Clojure macros. For example, it is possible to create macros that create recursive expansions that do not terminate and instead *diverge*, such as the following example:

```
(defmacro diverge [t]
  `((diverge ~t) (diverge ~t)))
#'user/diverge
```

```
user=> (diverge true)
Syntax error (StackOverflowError) compiling at (REPL:1:1).
null
```

As a final example, let's confirm that macros do in fact operate at compile time by constructing a macro that essentially acts as a closure that bridges from compile to runtime, shown next:

```
user=> (defmacro print-msg-with-compile []
  (let [num (System/currentTimeMillis)]
    `(fn [t#] (println t# " " ~num))))
#'user/print-msg-with-compile

user=> (def p1 (print-msg-with-compile))
#'user/p1

user=> (p1 "aaa")
aaa   1603437421852
nil

user=> (p1 "bbb")
bbb   1603437421852
nil
```

Notice how the `(let)` form is evaluated at compile time, so the value of `(System/ currentTimeMillis)` is captured when the macro is evaluated, bound to the symbol `num`, and then replaced in the expanded form with the value that was bound— effectively a constant determined at compile time.

Even though we have introduced macros at the very end of this chapter, macros are actually all around us in Clojure. In fact, much of the Clojure standard library is implemented as macros. The well-grounded developer can learn a lot by spending some time reading the source of the standard library and observing how key parts of it have been written.

At this point, a word of warning is also timely: macros are a powerful technique, and there is a temptation (just as there is with other techniques that "level up" a programmer's thinking) that some developers can fall prey to—the tendency to overuse the technique by including when it is not strictly necessary.

To guard against this, we highly recommend that you keep in mind the following simple general rules for the use of Clojure macros:

- Never write a macro when the goal can be accomplished with a function.
- Write a macro to implement a feature, capability, or pattern that is not already present in the language or standard library.

The first of these is, of course, merely the old adage that "just because you *can* do something doesn't mean that you *should* " in a different guise.

The second is a reminder that macros exist for a reason: there are things that you can do with them that cannot really be done in any other way. A proficient Clojure programmer will be able to use macros to great effect where appropriate.

Beyond macros, there is still more to learn about Clojure, such as the language's approach to dynamic runtime behavior. In Java this is usually handled using class and interface inheritance and virtual dispatch, but these are fundamentally object-oriented concepts and are not a particularly good fit for Clojure.

Instead, Clojure uses *protocols* and *datatypes*—along with the proxies that we have already met—to provide much of this flexibility. There are even more possibilities, such as custom dispatch schemes that use *multimethods*. These are also very powerful techniques but, unfortunately, are a little far outside of this introductory treatment of Clojure.

As a language, Clojure is arguably the most different from Java of the languages we've looked at. Its Lisp heritage, emphasis on immutability, and different approaches seem to make it into an entirely separate language. But its tight integration with the JVM, alignment of its type system (even when it provides alternatives, such as seqs), and the power of exploratory programming make it a very complementary language to Java.

The differences between the languages we've studied in this part clearly show the power of the Java platform to evolve and to continue to be a viable destination for application development. This is also a testament to the flexibility and capability of the JVM.

## Summary

- Clojure is dynamically typed, and Java programmers need to be careful of runtime exceptions.
- Exploratory and REPL-based development is a different feel from a Java IDE.
- Clojure provides and promotes a very immutable style of programming.

- Functional programming pervades Clojure—far more so than Java or Kotlin.
- Seqs are a functional equivalent to Java's iterators and collections.
- Macros define a compile-time transformation of Clojure source.

Previous chapter
< 9 Kotlin

Next chapter
Part 4. Build and deployment >